DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSN3001: Data Structures Lab
Lab Assignment: Stack Data Structure

1. Objectives & General Guidelines

Stacks are fundamental Last-In, First-Out (LIFO) data structures used for sequence reversal, expression evaluation, and nearest element lookup. This assignment covers basic stack verification, expression evaluation, and advanced monotonic stack techniques through three practical applications:

- **Syntax Balanced Parentheses Checker:** Utilizing a standard LIFO Stack to validate matching brackets in code expressions.
- **Postfix Expression Evaluator:** Utilizing an Operand Stack to evaluate Reverse Polish Notation arithmetic statements.
- **Stock Span Analysis Engine (Monotonic Stack):** Utilizing a Monotonic Decreasing Stack to compute consecutive price span bounds in linear time.

2. Lab Problems

Problem 1: Code Editor Syntax Balanced Parentheses Checker

Design a syntax verification engine for a code editor using a Stack data structure. Given a string containing characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid. An input string is valid if open brackets are closed by the same type of brackets and in the correct LIFO order.

Stack Logic & Algorithmic Architecture:

```
Stack Operations: Push open brackets '(', '{', '[' onto stack.
Match Check: On closing bracket ')', '}', ']', pop top element and verify matching pair.
Validation Rule: String is valid iff stack is empty at end and no mismatch occurs.
```

Sample Execution & System Interaction:

```
Input: expression = "{ [ ( ) ] }"
Output: Valid Syntax: True

Input: expression = "{ [ ( ] ) }"
Output: Valid Syntax: False (Mismatch at ']')
```

Constraints & Performance Bounds: *Length of expression string N: 1 <= N <= 10^5. Must execute in O(N) time and O(N) auxiliary space.*

Problem 2: Postfix Expression Evaluator (Reverse Polish Notation)

Design an expression calculation engine for a scientific calculator using an Operand Stack. Evaluate the value of an arithmetic expression presented in Postfix notation (Reverse Polish Notation). Valid operators are '+', '-', '*', and '/'. Each operand may be an integer, and division truncates toward zero.

Stack Logic & Algorithmic Architecture:

```
Stack Operations: Scan token array from left to right.
Operand Handling: If token is a number, push onto stack.
Operator Handling: If token is operator, pop b = stack.pop(), a = stack.pop(). Compute (a
op b) and push result.
Final Result: The remaining single element on the stack is the calculated output.
```

Sample Execution & System Interaction:

```
Input: tokens = ["2", "1", "+", "3", "*"]
Output: Result: 9  (Explanation: ((2 + 1) * 3) = 9)

Input: tokens = ["4", "13", "5", "/", "+"]
Output: Result: 6  (Explanation: (4 + (13 / 5)) = 6)
```

Constraints & Performance Bounds: *Number of tokens N: 1 <= N <= 10^4. Every token is an operator or integer in range [-200, 200].*

Problem 3: Financial Stock Span Analysis Engine (Monotonic Stack)

Design a daily stock market span tracking system using a Monotonic Stack. The span of a stock's price today is defined as the maximum number of consecutive days (including today) for which the price of the stock was less than or equal to today's price. Implement a dynamic class StockSpanner with method next(price) returning the calculated span for that day.

Stack Logic & Algorithmic Architecture:

```
Monotonic Stack Structure: Stack stores tuples (price, span).
next(price) Logic:
  1. span = 1
  2. While stack is not empty and stack.top().price <= price:
         span += stack.pop().span
  3. stack.push((price, span))
  4. Return span
```

Sample Execution & System Interaction:

```
Input Calls: next(100), next(80), next(60), next(70), next(60), next(75), next(85)
Output Spans: [1, 1, 1, 2, 1, 4, 6]
```

Constraints & Performance Bounds: *Number of price calls N: 1 <= N <= 10^5. Prices: 1 <= price <= 10^5. Amortized time complexity per call must be O(1).*